Sound + Motion ML Classifier

sound/motion setup with supervised `MLPClassifier` pipeline:

1. Load WAV and CSV files
2. Visualize sound and motion signals
3. Extract FFT bandpower and IMU magnitude features
4. Train `StandardScaler` + `MLPClassifier`
5. Evaluate with graphs and classification metrics
6. Export `weights_sound_motion.json` for Core2-style inference

Place the seven files in the same folder as this notebook before running:

`baseline.wav`, `humming.wav`, `vocal_distress.wav`, `sucking_teeth.wav`, `shaking.csv`, `tapping.csv`, `fidgeting.csv`

In [ ]:
```
# Sound + Motion ML Classifier
# Converted rule based F&Y code

# prerunning: place files in the same folder:
#   baseline.wav, humming.wav, vocal_distress.wav, sucking_teeth.wav
#   shaking.csv, tapping.csv, fidgeting.csv

# Output:
#   weights_sound_motion.json
```

In [ ]:
```
# Section 1: Setup

import os
import json
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
from matplotlib.colors import LinearSegmentedColormap

from scipy.io import wavfile
from scipy.signal import spectrogram as sci_spectrogram
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import confusion_matrix, classification_report

plt.rcParams.update({
    "figure.figsize": (12, 5),
```

```
    "figure.dpi": 100,
    "axes.grid": True,
    "grid.alpha": 0.25,
    "axes.edgecolor": "#D9EAF7",
    "axes.labelcolor": "#16324F",
    "text.color": "#16324F",
    "xtick.color": "#16324F",
    "ytick.color": "#16324F",
    "axes.spines.top": False,
    "axes.spines.right": False,
})

#Main variables
sounds = {
    "baseline": "baseline.wav",
    "humming": "humming.wav",
    "vocal_distress": "vocal_distress.wav",
    "sucking_teeth": "sucking_teeth.wav",
}

motions = {
    "shaking": "shaking.csv",
    "tapping": "tapping.csv",
    "fidgeting": "fidgeting.csv",
}

# Stress level mapping pipeline retained from rule based F&Y_code (thi
stress_level_map = {
    "baseline": "level_0_no_stress",
    "humming": "level_1_sound_stress",
    "vocal_distress": "level_1_sound_stress",
    "sucking_teeth": "level_1_sound_stress",
    "shaking": "level_2_motion_stress",
    "tapping": "level_2_motion_stress",
    "fidgeting": "level_2_motion_stress",
}

# Blue + green graph theme
THEME_BLUE = "#2171B5"
THEME_GREEN = "#238B45"
THEME_DARK_BLUE = "#084594"
THEME_DARK_GREEN = "#006D2C"
THEME_LIGHT_BLUE = "#9ECAE1"
THEME_LIGHT_GREEN = "#A1D99B"
THEME_PALETTE = [
    "#C6DBEF", "#6BAED6", "#2171B5", "#084594",
    "#C7E9C0", "#74C476", "#238B45", "#006D2C"
]
CMAP_BLUE_GREEN = LinearSegmentedColormap.from_list(
    "blue_green_theme",
    ["#EFF3FF", "#BDD7E7", "#6BAED6", "#31A354", "#006D2C"]
)
```

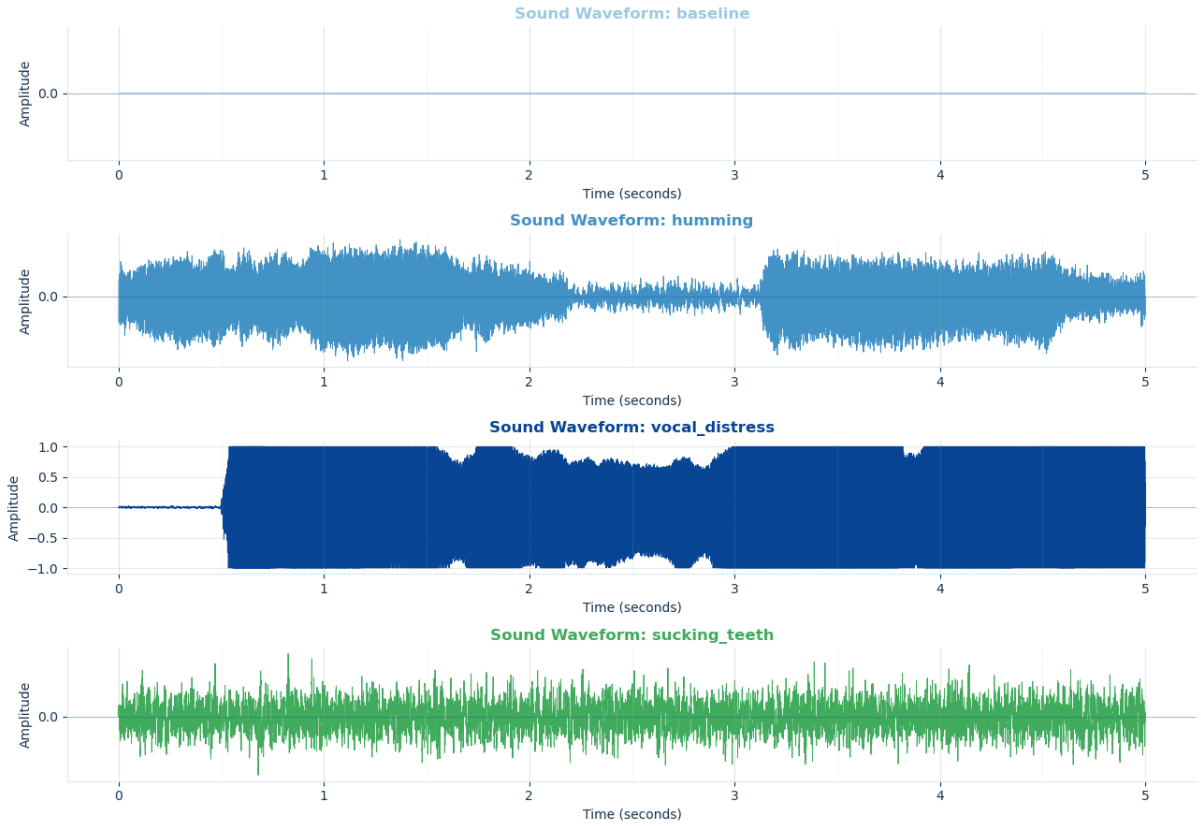
```
COLORS = {
    "baseline": "#9ECAE1",
    "humming": "#4292C6",
    "vocal_distress": "#084594",
    "sucking_teeth": "#41AB5D",
    "shaking": "#238B45",
    "tapping": "#006D2C",
    "fidgeting": "#00441B",
}

SAMPLE_RATE = 8000
IMU_RATE = 50
WIN_SEC = 0.5
WIN_SAMPLES = int(SAMPLE_RATE * WIN_SEC)
MOTION_WIN_ROWS = int(IMU_RATE * WIN_SEC)
KEEP_SEC = None

# Module Train-style FFT setup
SUB_N = 256
N_SUB = WIN_SAMPLES // SUB_N
SUB_HZ = SAMPLE_RATE / SUB_N

BANDS = [
    (0, 300),
    (300, 1000),
    (1000, 2500),
    (2500, 4000),
    (0, 4000),
]
AUDIO_FEATURE_NAMES = [
    "audio_bp_0_300",
    "audio_bp_300_1000",
    "audio_bp_1000_2500",
    "audio_bp_2500_4000",
    "audio_bp_full",
]
MOTION_FEATURE_NAMES = [
    "accel_mean", "accel_std", "accel_max", "accel_range", "accel_ener
    "gyro_mean", "gyro_std", "gyro_max", "gyro_range", "gyro_energy",
]
FEATURE_NAMES = AUDIO_FEATURE_NAMES + MOTION_FEATURE_NAMES

print("Setup complete.")
print(f"Sound window: {WIN_SAMPLES} samples = {WIN_SEC:.2f} seconds")
print(f"Motion window: {MOTION_WIN_ROWS} rows = {WIN_SEC:.2f} seconds"
print(f"Feature count: {len(FEATURE_NAMES)}")
```

```
Setup complete.
Sound window: 4000 samples = 0.50 seconds
Motion window: 25 rows = 0.50 seconds
Feature count: 15
```

In [ ]:
```
# Section 2: Load sound and motion files

def check_files(file_dict):
    missing = [path for path in file_dict.values() if not os.path.exis
    if missing:
        print("Missing files:")
        for path in missing:
            print(" -", path)
        raise FileNotFoundError("Place all WAV and CSV files in the sa

check_files(sounds)
check_files(motions)

def load_wav(path, keep_sec=None):
    """Load WAV as int16 and float copy, matching the Module Train pat
    sr, raw = wavfile.read(path)
    if raw.ndim > 1:
        raw = raw[:, 0]
    if sr != SAMPLE_RATE:
        raise ValueError(f"{path}: expected {SAMPLE_RATE} Hz, got {sr}
    if keep_sec is not None:
        raw = raw[:int(keep_sec * sr)]
    raw_i16 = raw.astype(np.int16)
    raw_f32 = raw_i16.astype(np.float64) / 32768.0
    return raw_i16, raw_f32

def load_motion(path, keep_sec=None):
    """Load motion CSV and make sure required IMU columns exist."""
    df = pd.read_csv(path)
    required = ["ax", "ay", "az", "gx", "gy", "gz"]
    missing = [col for col in required if col not in df.columns]
    if missing:
        raise ValueError(f"{path} is missing columns: {missing}")
    df = df[required].copy()
    if keep_sec is not None:
        df = df.iloc[:int(keep_sec * IMU_RATE)].copy()
    return df

recordings_i16 = {}
recordings_f32 = {}
motion_data = {}

for label, path in sounds.items():
    i16, f32 = load_wav(path, keep_sec=KEEP_SEC)
    recordings_i16[label] = i16
    recordings_f32[label] = f32
    print(f"{label:15s} sound  {len(i16):6d} samples  {len(i16)/SAMPLE

for label, path in motions.items():
    df = load_motion(path, keep_sec=KEEP_SEC)
    motion_data[label] = df
    print(f"{label:15s} motion {len(df):6d} rows     {len(df)/IMU_RATE
```

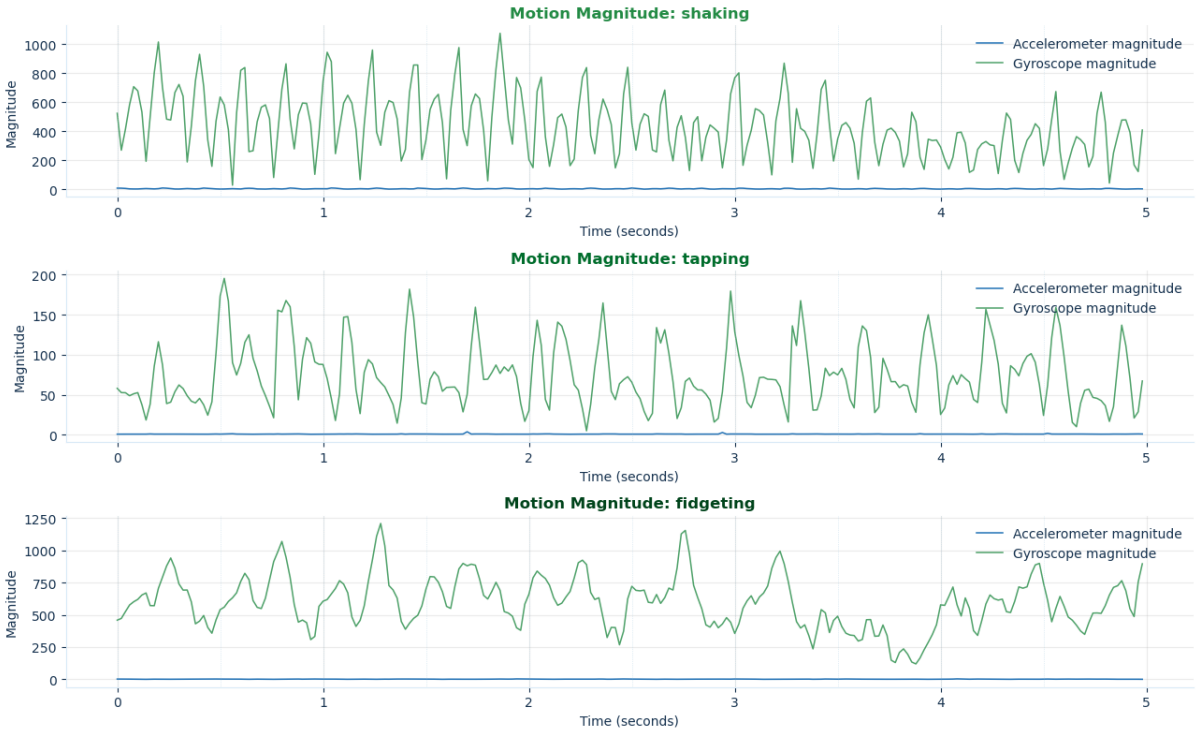
```
baseline        sound   40000 samples  5.0 s
humming         sound   40000 samples  5.0 s
vocal_distress  sound   40000 samples  5.0 s
sucking_teeth   sound   40000 samples  5.0 s
shaking         motion    250 rows     5.0 s
tapping         motion    250 rows     5.0 s
fidgeting       motion    250 rows     5.0 s
```

In [ ]:
```python
# Section 3 Graphs: time series waveforms

fig, axes = plt.subplots(len(recordings_f32), 1, figsize=(13, 9), shar
if len(recordings_f32) == 1:
    axes = [axes]

for ax, (label, audio) in zip(axes, recordings_f32.items()):
    t = np.arange(len(audio)) / SAMPLE_RATE
    ax.plot(t, audio, color=COLORS[label], linewidth=0.7)
    for k in np.arange(0, t[-1], WIN_SEC):
        ax.axvline(k, color=THEME_LIGHT_GREEN, linewidth=0.5, linestyl
    ax.axhline(0, color=THEME_DARK_BLUE, linewidth=0.6, alpha=0.35)
    ax.set_title(f"Sound Waveform: {label}", color=COLORS[label], font
    ax.set_xlabel("Time (seconds)")
    ax.set_ylabel("Amplitude")
    ax.yaxis.set_major_locator(mticker.MultipleLocator(0.5))

plt.tight_layout()
plt.show()
```



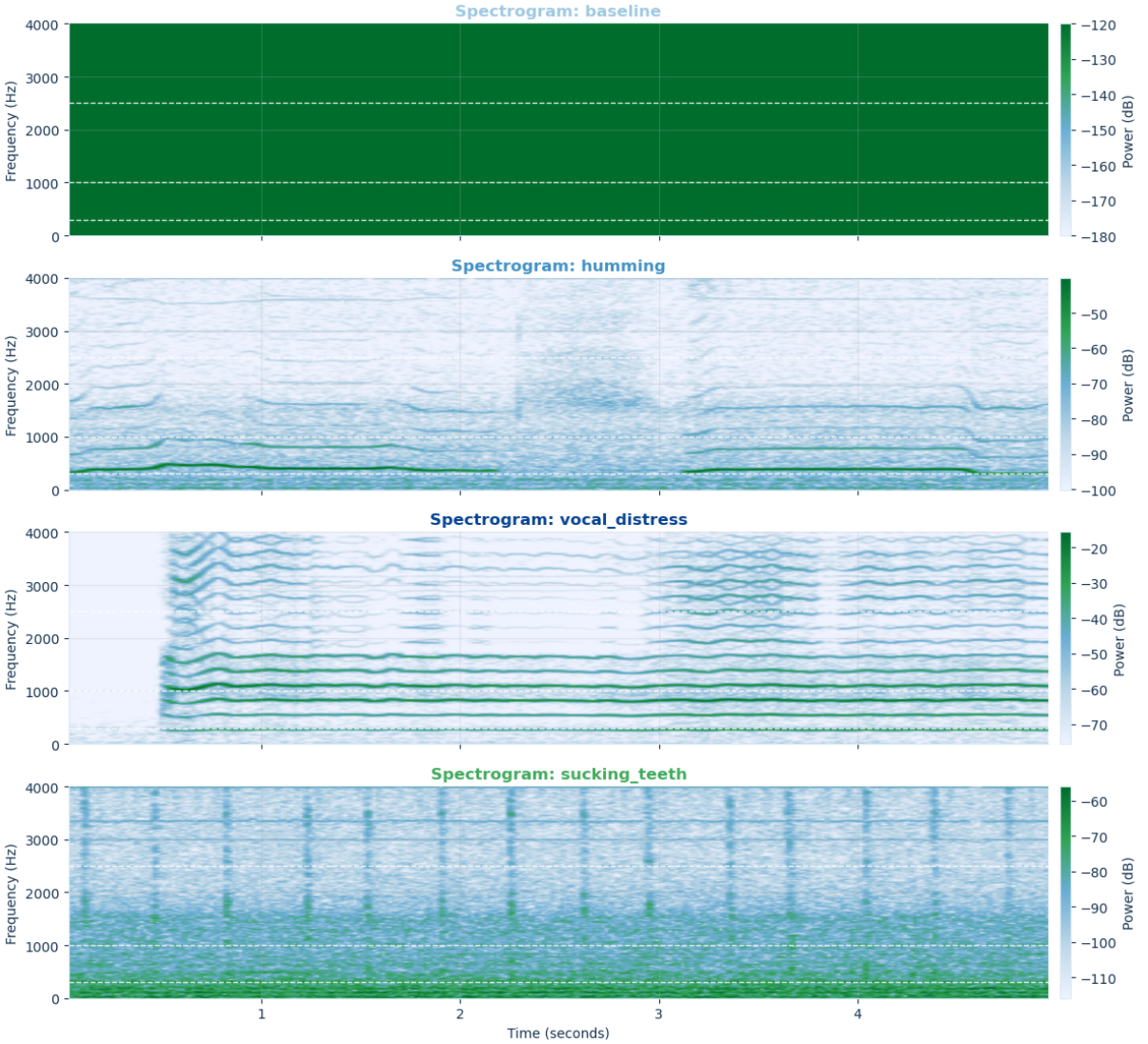
In [ ]:
```python
# Section 4 Graphs: motion magnitude
```

```
fig, axes = plt.subplots(len(motion_data), 1, figsize=(13, 8), sharex=
if len(motion_data) == 1:
    axes = [axes]

for ax, (label, df) in zip(axes, motion_data.items()):
    t = np.arange(len(df)) / IMU_RATE
    accel_mag = np.sqrt(df["ax"]**2 + df["ay"]**2 + df["az"]**2)
    gyro_mag = np.sqrt(df["gx"]**2 + df["gy"]**2 + df["gz"]**2)
    ax.plot(t, accel_mag, linewidth=1.1, label="Accelerometer magnitud
    ax.plot(t, gyro_mag, linewidth=1.1, label="Gyroscope magnitude", c
    for k in np.arange(0, t[-1], WIN_SEC):
        ax.axvline(k, color=THEME_LIGHT_BLUE, linewidth=0.5, linestyle
    ax.set_title(f"Motion Magnitude: {label}", color=COLORS[label], fo
    ax.set_xlabel("Time (seconds)")
    ax.set_ylabel("Magnitude")
    ax.legend(frameon=False)

plt.tight_layout()
plt.show()
```



In [ ]:
```
# Section 5 Graphs: sound spectrograms

BAND_BOUNDARY = [300, 1000, 2500]
fig, axes = plt.subplots(len(recordings_f32), 1, figsize=(13, 11), sha
if len(recordings_f32) == 1:
    axes = [axes]

for ax, (label, audio) in zip(axes, recordings_f32.items()):
    f_bins, t_bins, Sxx = sci_spectrogram(
        audio, fs=SAMPLE_RATE, nperseg=512, noverlap=384, window="hann
```

```
    )
    Sxx_db = 10 * np.log10(Sxx + 1e-12)
    vmax = Sxx_db.max()
    im = ax.pcolormesh(
        t_bins, f_bins, Sxx_db,
        shading="gouraud", vmin=vmax-60, vmax=vmax,
        cmap=CMAP_BLUE_GREEN
    )
    for fb in BAND_BOUNDARY:
        ax.axhline(fb, color="white", linewidth=1.0, linestyle="--", a
    ax.set_ylim(0, SAMPLE_RATE // 2)
    ax.set_ylabel("Frequency (Hz)")
    ax.set_title(f"Spectrogram: {label}", color=COLORS[label], fontwei
    fig.colorbar(im, ax=ax, label="Power (dB)", pad=0.01)

axes[-1].set_xlabel("Time (seconds)")
plt.tight_layout()
plt.show()
```



In [ ]:
```
# Section 6: Feature extraction
```

```
def audio_bandpower_features(window_int16):
    """Module Train-style 5-band FFT bandpower for one sound window."""
    bp = np.zeros(len(BANDS), dtype=np.float64)
    for w in range(N_SUB):
        sub = window_int16[w * SUB_N : (w + 1) * SUB_N].astype(np.floa
        Xf = np.fft.rfft(sub)
        mag_sq = Xf.real**2 + Xf.imag**2
        for i, (f_lo, f_hi) in enumerate(BANDS):
            lo = int(f_lo / SUB_HZ)
            hi = min(int(f_hi / SUB_HZ), len(mag_sq))
            bp[i] += float(np.sum(mag_sq[lo:hi])) / SUB_N
    return list(bp / N_SUB)

def motion_features(window_df):
    """Motion features for one 0.5-second IMU window."""
    accel_mag = np.sqrt(window_df["ax"]**2 + window_df["ay"]**2 + wind
    gyro_mag = np.sqrt(window_df["gx"]**2 + window_df["gy"]**2 + windo
    return [
        float(np.mean(accel_mag)),
        float(np.std(accel_mag)),
        float(np.max(accel_mag)),
        float(np.max(accel_mag) - np.min(accel_mag)),
        float(np.mean(accel_mag**2)),
        float(np.mean(gyro_mag)),
        float(np.std(gyro_mag)),
        float(np.max(gyro_mag)),
        float(np.max(gyro_mag) - np.min(gyro_mag)),
        float(np.mean(gyro_mag**2)),
    ]

def extract_sound_rows(recordings_i16):
    rows = []
    for label, audio in recordings_i16.items():
        n_windows = len(audio) // WIN_SAMPLES
        for k in range(n_windows):
            win = audio[k * WIN_SAMPLES : (k + 1) * WIN_SAMPLES]
            audio_feats = audio_bandpower_features(win)
            motion_feats = [0.0] * len(MOTION_FEATURE_NAMES)
            rows.append({
                "label": label,
                "stress_level": stress_level_map[label],
                "source_type": "sound",
                "window": k,
                **dict(zip(FEATURE_NAMES, audio_feats + motion_feats))
            })
        print(f"{label:15s} sound windows : {n_windows}")
    return rows

def extract_motion_rows(motion_data):
    rows = []
    for label, df in motion_data.items():
        n_windows = len(df) // MOTION_WIN_ROWS
```

```
        for k in range(n_windows):
            win = df.iloc[k * MOTION_WIN_ROWS : (k + 1) * MOTION_WIN_R
            audio_feats = [0.0] * len(AUDIO_FEATURE_NAMES)
            motion_feats = motion_features(win)
            rows.append({
                "label": label,
                "stress_level": stress_level_map[label],
                "source_type": "motion",
                "window": k,
                **dict(zip(FEATURE_NAMES, audio_feats + motion_feats))
            })
        print(f"{label:15s} motion windows: {n_windows}")
    return rows

rows = extract_sound_rows(recordings_i16) + extract_motion_rows(motion
features_df = pd.DataFrame(rows)

X = features_df[FEATURE_NAMES].to_numpy(dtype=np.float64)
y = features_df["label"].to_numpy()

print()
print("Feature table shape:", features_df.shape)
print("X shape:", X.shape)
print("Class balance:")
print(features_df["label"].value_counts())

features_df.head()
```

```
baseline        sound windows : 10
humming         sound windows : 10
vocal_distress  sound windows : 10
sucking_teeth   sound windows : 10
shaking         motion windows: 10
tapping         motion windows: 10
fidgeting       motion windows: 10

Feature table shape: (70, 19)
X shape: (70, 15)
Class balance:
label
baseline          10
humming           10
vocal_distress    10
sucking_teeth     10
shaking           10
tapping           10
fidgeting         10
Name: count, dtype: int64
```

Out[ ]:

	label	stress_level	source_type	window	audio_bp_0_300	audio_bp
0	baseline	level_0_no_stress	sound	0	0.0	
1	baseline	level_0_no_stress	sound	1	0.0	
2	baseline	level_0_no_stress	sound	2	0.0	
3	baseline	level_0_no_stress	sound	3	0.0	
4	baseline	level_0_no_stress	sound	4	0.0	

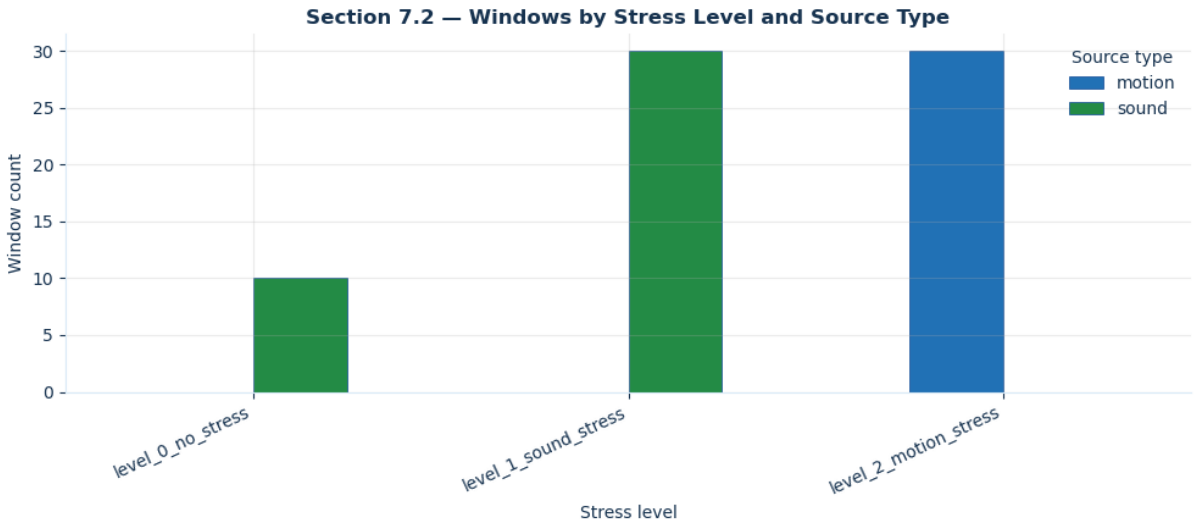
In [ ]:

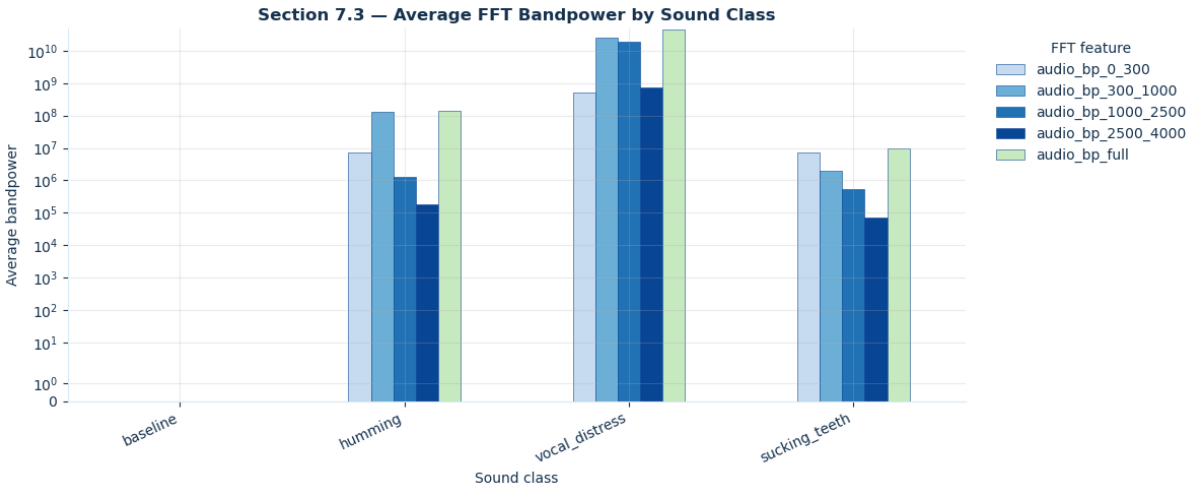
```
# Section 7 Detailed Graphs: training balance, feature patterns, and m

# 7.1: Training window count by class
CLASS_ORDER = list(sounds.keys()) + list(motions.keys())
class_counts = features_df["label"].value_counts().reindex(CLASS_ORDER

fig, ax = plt.subplots(figsize=(11, 4.5))
ax.bar(class_counts.index, class_counts.values, color=[COLORS[c] for c
ax.set_title("Section 7.1 — Training Windows by Class", fontweight="bo
ax.set_xlabel("Class")
ax.set_ylabel("Window count")
ax.set_xticklabels(class_counts.index, rotation=35, ha="right")
for i, v in enumerate(class_counts.values):
    ax.text(i, v + max(class_counts.values) * 0.02, int(v), ha="center
plt.tight_layout()
plt.show()

# 7.2: Windows by source type and stress level
source_stress = pd.crosstab(features_df["stress_level"], features_df["
source_stress = source_stress.reindex(sorted(source_stress.index))

fig, ax = plt.subplots(figsize=(10, 4.5))
source_stress.plot(kind="bar", ax=ax, color=[THEME_BLUE, THEME_GREEN],
ax.set_title("Section 7.2 — Windows by Stress Level and Source Type",
ax.set_xlabel("Stress level")
ax.set_ylabel("Window count")
ax.legend(title="Source type", frameon=False)
plt.xticks(rotation=25, ha="right")
plt.tight_layout()
plt.show()

# 7.3: Average audio FFT bandpower by sound class
sound_means = features_df[features_df["source_type"] == "sound"].group

fig, ax = plt.subplots(figsize=(12, 5))
sound_means.plot(kind="bar", ax=ax, color=THEME_PALETTE[:len(AUDIO_FEA
ax.set_title("Section 7.3 — Average FFT Bandpower by Sound Class", fon
ax.set_xlabel("Sound class")
ax.set_ylabel("Average bandpower")
ax.set_yscale("symlog")
```

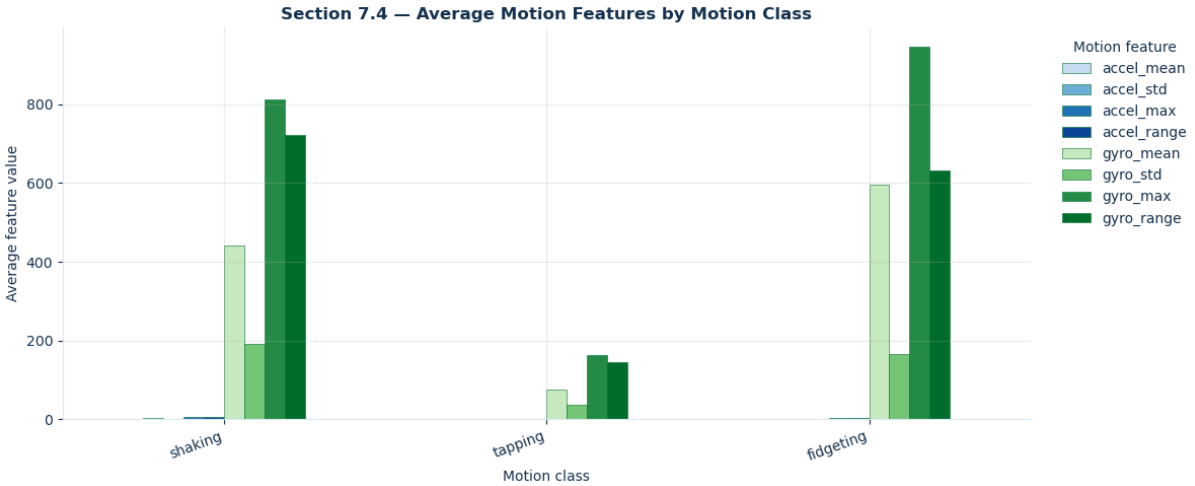
```
ax.legend(title="FFT feature", bbox_to_anchor=(1.02, 1), loc="upper le
plt.xticks(rotation=25, ha="right")
plt.tight_layout()
plt.show()

# 7.4: Average motion magnitude features by motion class
motion_focus = ["accel_mean", "accel_std", "accel_max", "accel_range",
motion_means = features_df[features_df["source_type"] == "motion"].gro

fig, ax = plt.subplots(figsize=(12, 5))
motion_means.plot(kind="bar", ax=ax, color=THEME_PALETTE[:len(motion_f
ax.set_title("Section 7.4 — Average Motion Features by Motion Class",
ax.set_xlabel("Motion class")
ax.set_ylabel("Average feature value")
ax.legend(title="Motion feature", bbox_to_anchor=(1.02, 1), loc="upper
plt.xticks(rotation=20, ha="right")
plt.tight_layout()
plt.show()

# 7.5: Raw feature heatmap by class
feature_means = features_df.groupby("label")[FEATURE_NAMES].mean().rei

fig, ax = plt.subplots(figsize=(13, 5.5))
im = ax.imshow(feature_means, aspect="auto", cmap=CMAP_BLUE_GREEN)
fig.colorbar(im, ax=ax, label="Average raw feature value")
ax.set_yticks(range(len(feature_means.index)))
ax.set_yticklabels(feature_means.index)
ax.set_xticks(range(len(FEATURE_NAMES)))
ax.set_xticklabels(FEATURE_NAMES, rotation=45, ha="right")
ax.set_title("Section 7.5 — Raw Feature Heatmap by Class", fontweight=
plt.tight_layout()
plt.show()

# 7.6: Scaled feature heatmap by class, using the same StandardScaler
section7_scaler = StandardScaler()
scaled_preview = section7_scaler.fit_transform(features_df[FEATURE_NAM
scaled_df = pd.DataFrame(scaled_preview, columns=FEATURE_NAMES)
scaled_df["label"] = features_df["label"].values
scaled_means = scaled_df.groupby("label")[FEATURE_NAMES].mean().reinde

fig, ax = plt.subplots(figsize=(13, 5.5))
limit = float(np.nanmax(np.abs(scaled_means.to_numpy())))
im = ax.imshow(scaled_means, aspect="auto", cmap=CMAP_BLUE_GREEN, vmin
fig.colorbar(im, ax=ax, label="Average scaled feature value")
ax.set_yticks(range(len(scaled_means.index)))
ax.set_yticklabels(scaled_means.index)
ax.set_xticks(range(len(FEATURE_NAMES)))
ax.set_xticklabels(FEATURE_NAMES, rotation=45, ha="right")
ax.set_title("Section 7.6 — Scaled Feature Heatmap by Class", fontweig
plt.tight_layout()
plt.show()
```

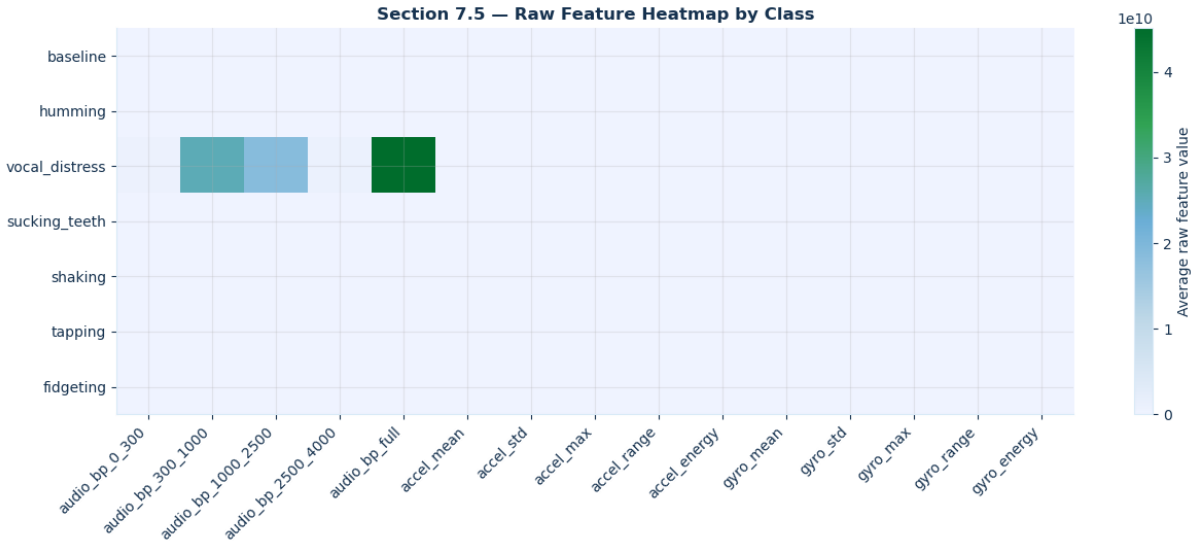
```
# 7.7: Feature correlation map to inspect redundancy before ML trainin
corr = features_df[FEATURE_NAMES].corr().fillna(0)

fig, ax = plt.subplots(figsize=(10, 8))
im = ax.imshow(corr, cmap=CMAP_BLUE_GREEN, vmin=-1, vmax=1)
fig.colorbar(im, ax=ax, label="Correlation")
ax.set_xticks(range(len(FEATURE_NAMES)))
ax.set_yticks(range(len(FEATURE_NAMES)))
ax.set_xticklabels(FEATURE_NAMES, rotation=45, ha="right")
ax.set_yticklabels(FEATURE_NAMES)
ax.set_title("Section 7.7 — Feature Correlation Map", fontweight="bold
plt.tight_layout()
plt.show()

# 7.8: Most variable features after scaling, useful for explaining whi
feature_spread = scaled_means.std(axis=0).sort_values(ascending=False)

fig, ax = plt.subplots(figsize=(11, 4.5))
ax.bar(feature_spread.index, feature_spread.values, color=THEME_GREEN,
ax.set_title("Section 7.8 — Feature Separation Strength Across Classes
ax.set_xlabel("Feature")
ax.set_ylabel("Standard deviation of class means")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```

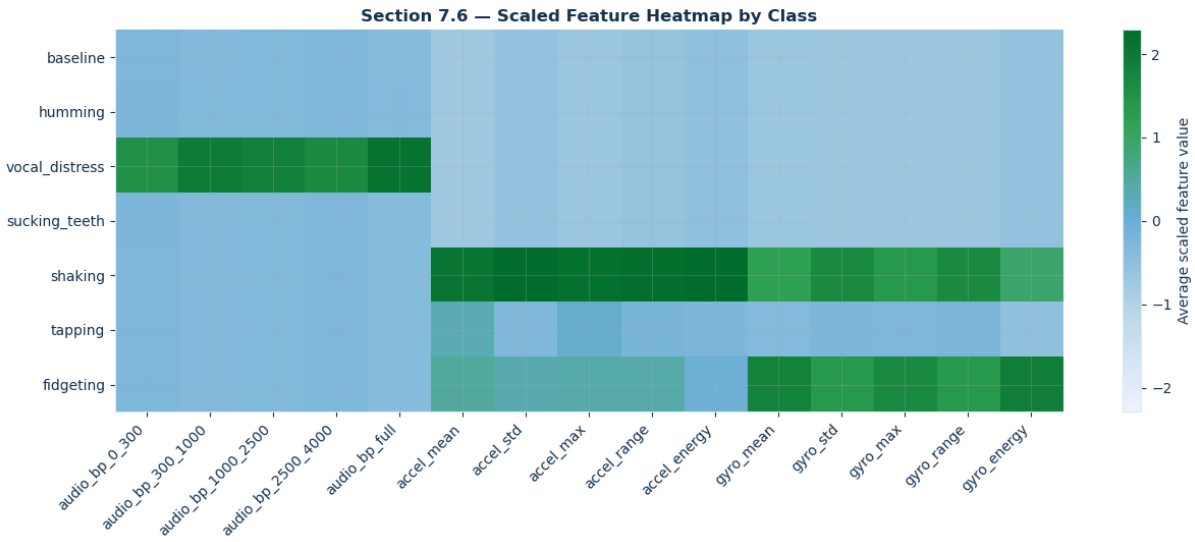
```
/var/folders/rk/yr9hvy2j00z9xy39ml6pq8mw0000gn/T/ipykernel_6169/3699979
882.py:11: UserWarning: set_ticklabels() should only be used with a fix
ed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels(class_counts.index, rotation=35, ha="right")
```

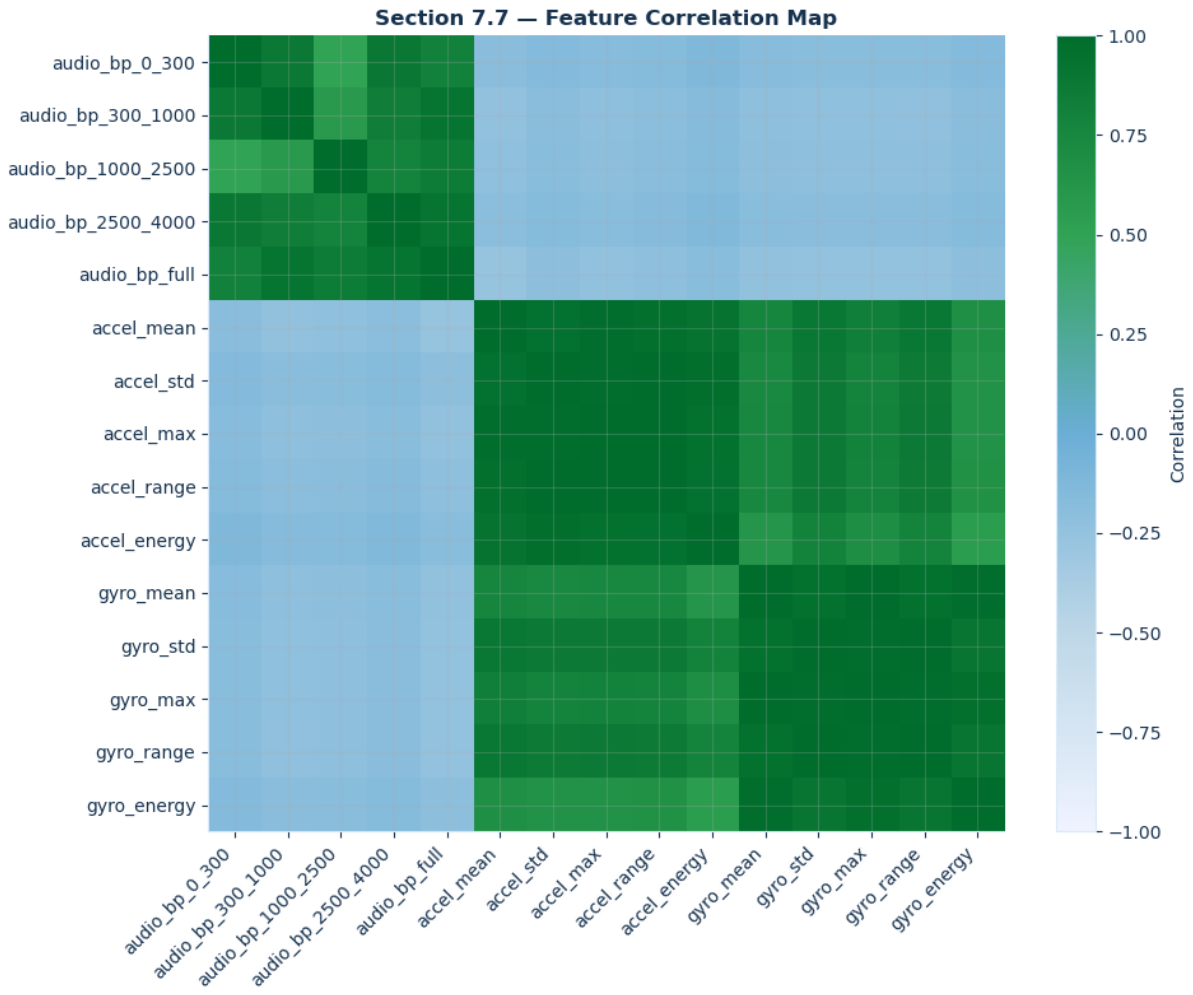

Section 7.2 — Windows by Stress Level and Source Type
Window count
Stress level
Source type
motion
sound
level_0_no_stress
level_1_sound_stress
level_2_motion_stress

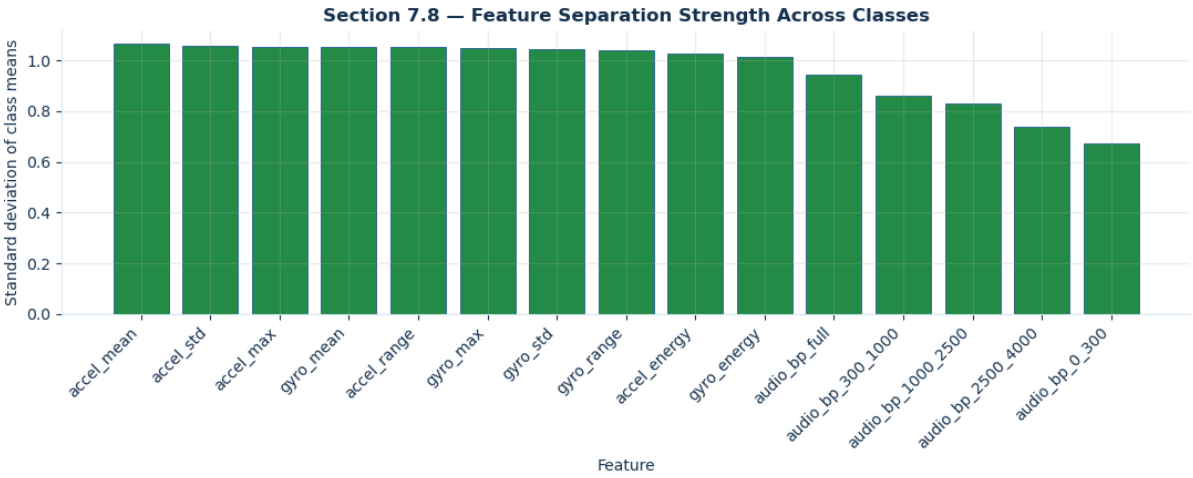

Section 7.3 — Average FFT Bandpower by Sound Class
Average bandpower
Sound class
FFT feature
audio_bp_0_300
audio_bp_300_1000
audio_bp_1000_2500
audio_bp_2500_4000
audio_bp_full
baseline
humming
vocal_distress
sucking_teeth


Section 7.4 — Average Motion Features by Motion Class
Average feature value
Motion class
Motion feature
accel_mean
accel_std
accel_max
accel_range
gyro_mean
gyro_std
gyro_max
gyro_range
shaking
tapping
fidgeting


Section 7.5 — Raw Feature Heatmap by Class
baseline
humming
vocal_distress
sucking_teeth
shaking
tapping
fidgeting
audio_bp_0_300
audio_bp_300_1000
audio_bp_1000_2500
audio_bp_2500_4000
audio_bp_full
accel_mean
accel_std
accel_max
accel_range
accel_energy
gyro_mean
gyro_std
gyro_max
gyro_range
gyro_energy
1e10
4
3
2
1
0
Average raw feature value


Section 7.6 — Scaled Feature Heatmap by Class
baseline
humming
vocal_distress
sucking_teeth
shaking
tapping
fidgeting
audio_bp_0_300
audio_bp_300_1000
audio_bp_1000_2500
audio_bp_2500_4000
audio_bp_full
accel_mean
accel_std
accel_max
accel_range
accel_energy
gyro_mean
gyro_std
gyro_max
gyro_range
gyro_energy
2
1
0
−1
−2
Average scaled feature value





In [ ]:
```
# Section 8 accuarcy, trianing and loss
# For very small class counts, stratify only when every class has at l
class_counts = pd.Series(y).value_counts()
stratify_y = y if class_counts.min() >= 2 else None

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=stratify_y, random_state=42
)

scaler = StandardScaler()
```

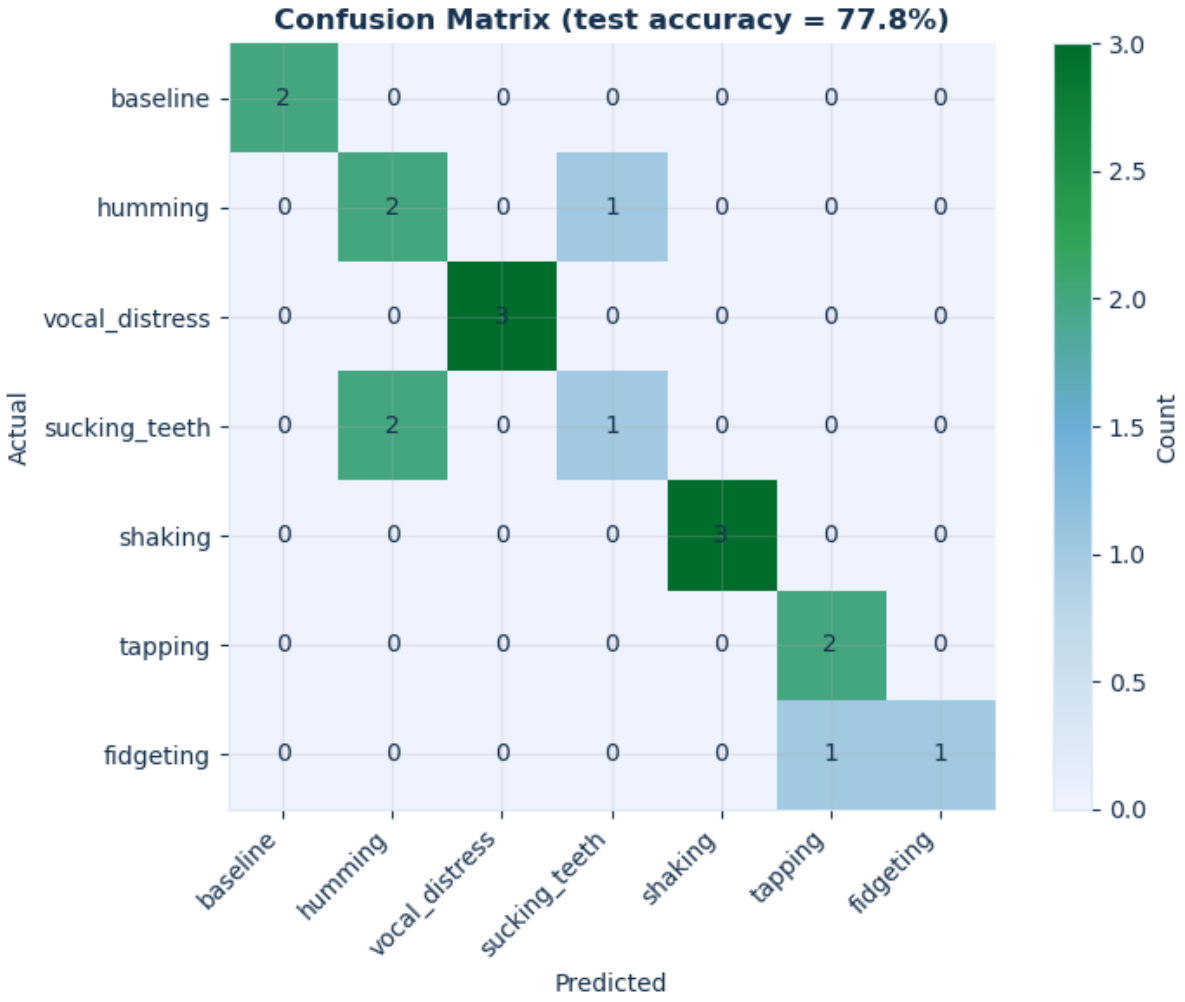
```
Xtr = scaler.fit_transform(X_train)
Xte = scaler.transform(X_test)

clf = MLPClassifier(
    hidden_layer_sizes=(16, 8),
    activation="relu",
    solver="adam",
    alpha=1e-3,
    max_iter=1000,
    random_state=42,
)
clf.fit(Xtr, y_train)

y_pred = clf.predict(Xte)
train_acc = clf.score(Xtr, y_train)
test_acc = clf.score(Xte, y_test)

print(f"Training iterations : {clf.n_iter_}")
print(f"Final loss          : {clf.loss_:.4f}")
print(f"Training accuracy   : {train_acc:.1%}")
print(f"Test accuracy       : {test_acc:.1%}")
print()
print(classification_report(y_test, y_pred, zero_division=0))
```

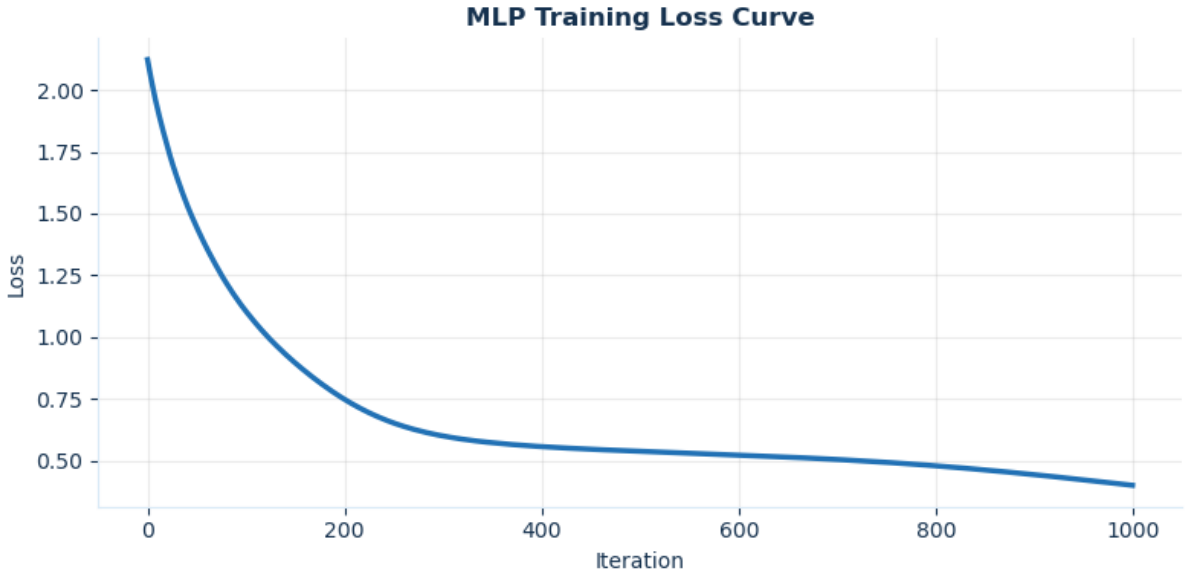
```
Training iterations : 1000
Final loss          : 0.3996
Training accuracy   : 86.5%
Test accuracy       : 77.8%

                precision    recall  f1-score   support

      baseline       1.00      1.00      1.00         2
     fidgeting       1.00      0.50      0.67         2
       humming       0.50      0.67      0.57         3
       shaking       1.00      1.00      1.00         3
 sucking_teeth       0.50      0.33      0.40         3
       tapping       0.67      1.00      0.80         2
vocal_distress       1.00      1.00      1.00         3

      accuracy                           0.78        18
     macro avg       0.81      0.79      0.78        18
  weighted avg       0.80      0.78      0.77        18
```

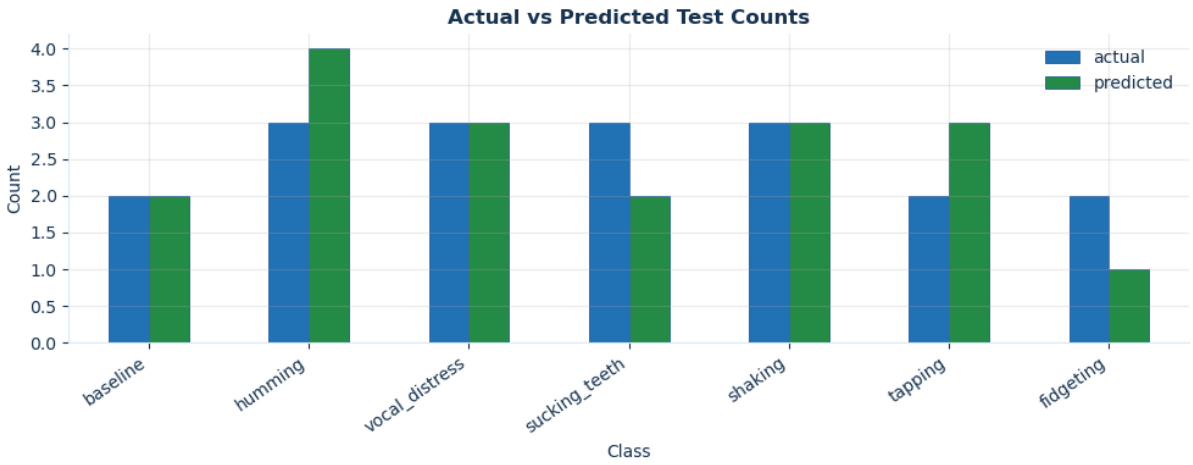
```
/opt/anaconda3/lib/python3.13/site-packages/sklearn/neural_network/_mul
tilayer_perceptron.py:785: ConvergenceWarning: Stochastic Optimizer: Ma
ximum iterations (1000) reached and the optimization hasn't converged y
et.
  warnings.warn(
```

In [ ]:
```
# Section 9 Graphs: confusion matrix and loss curve

CLASS_ORDER = list(sounds.keys()) + list(motions.keys())
cm = confusion_matrix(y_test, y_pred, labels=CLASS_ORDER)
```

```
fig, ax = plt.subplots(figsize=(8, 6))
im = ax.imshow(cm, cmap=CMAP_BLUE_GREEN)
plt.colorbar(im, ax=ax, label="Count")
ax.set_xticks(range(len(CLASS_ORDER)))
ax.set_yticks(range(len(CLASS_ORDER)))
ax.set_xticklabels(CLASS_ORDER, rotation=45, ha="right")
ax.set_yticklabels(CLASS_ORDER)
ax.set_xlabel("Predicted")
ax.set_ylabel("Actual")
ax.set_title(f"Confusion Matrix (test accuracy = {test_acc:.1%})", fon
for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        ax.text(j, i, cm[i, j], ha="center", va="center", color="#1632
plt.tight_layout()
plt.show()

plt.figure(figsize=(8, 4))
plt.plot(clf.loss_curve_, linewidth=2.5, color=THEME_BLUE)
plt.title("MLP Training Loss Curve", fontweight="bold")
plt.xlabel("Iteration")
plt.ylabel("Loss")
plt.tight_layout()
plt.show()

# Actual vs predicted counts
count_compare = pd.DataFrame({
    "actual": pd.Series(y_test).value_counts(),
    "predicted": pd.Series(y_pred).value_counts(),
}).fillna(0).reindex(CLASS_ORDER).fillna(0)

ax = count_compare.plot(kind="bar", figsize=(10, 4), color=[THEME_BLUE
ax.set_title("Actual vs Predicted Test Counts", fontweight="bold")
ax.set_xlabel("Class")
ax.set_ylabel("Count")
ax.legend(frameon=False)
plt.xticks(rotation=35, ha="right")
plt.tight_layout()
plt.show()
```


Confusion Matrix (test accuracy = 77.8%)
baseline
humming
vocal_distress
sucking_teeth
shaking
tapping
fidgeting
Actual
Predicted
Count
3.0
2.5
2.0
1.5
1.0
0.5
0.0


MLP Training Loss Curve
Loss
Iteration
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0
200
400
600
800
1000



In [ ]:
```python
# Section 10: Export weights JSON for Core2-style inference

weights = {
    "coefs": [W.tolist() for W in clf.coefs_],
    "intercepts": [b.tolist() for b in clf.intercepts_],
    "classes": [str(c) for c in clf.classes_],
    "feature_names": FEATURE_NAMES,
    "audio_feature_names": AUDIO_FEATURE_NAMES,
    "motion_feature_names": MOTION_FEATURE_NAMES,
    "scaler_mean": scaler.mean_.tolist(),
    "scaler_std": scaler.scale_.tolist(),
    "stress_level_map": stress_level_map,
    "sample_rate": SAMPLE_RATE,
    "imu_rate": IMU_RATE,
    "win_sec": WIN_SEC,
    "sub_n": SUB_N,
    "bands": BANDS,
}

with open("weights_sound_motion.json", "w") as f:
    json.dump(weights, f)

print("Saved: weights_sound_motion.json")
print()
print("Layer shapes:")
for i, (W, b) in enumerate(zip(clf.coefs_, clf.intercepts_)):
    print(f"  Layer {i+1}: W {list(W.shape)}   b {list(b.shape)}")
print("Classes:", weights["classes"])
print("Feature names:", FEATURE_NAMES)
```

```
Saved: weights_sound_motion.json

Layer shapes:
  Layer 1: W [15, 16]   b [16]
  Layer 2: W [16, 8]   b [8]
  Layer 3: W [8, 7]   b [7]
Classes: ['baseline', 'fidgeting', 'humming', 'shaking', 'sucking_teet
h', 'tapping', 'vocal_distress']
Feature names: ['audio_bp_0_300', 'audio_bp_300_1000', 'audio_bp_1000_2
500', 'audio_bp_2500_4000', 'audio_bp_full', 'accel_mean', 'accel_std',
'accel_max', 'accel_range', 'accel_energy', 'gyro_mean', 'gyro_std', 'g
yro_max', 'gyro_range', 'gyro_energy']
```

In [ ]:

```python
# Section 11 Output verification: pure Python forward pass

with open("weights_sound_motion.json", "r") as f:
    md = json.load(f)

W_py = md["coefs"]
b_py = md["intercepts"]
CLASSES_PY = md["classes"]
SC_MEAN = md["scaler_mean"]
SC_STD = md["scaler_std"]

def scale_manual(x, mean, std):
    return [(x[i] - mean[i]) / std[i] for i in range(len(x))]

def dot_manual(x, W, b):
    result = []
    for j in range(len(W[0])):
        total = b[j]
        for i in range(len(x)):
            total += x[i] * W[i][j]
        result.append(total)
    return result

def relu_manual(x):
    return [max(0.0, v) for v in x]

def softmax_manual(z):
    m = max(z)
    e = [math.exp(v - m) for v in z]
    s = sum(e)
    return [v / s for v in e]

def predict_manual(raw_features):
    x = scale_manual(raw_features, SC_MEAN, SC_STD)
    h1 = relu_manual(dot_manual(x, W_py[0], b_py[0]))
    h2 = relu_manual(dot_manual(h1, W_py[1], b_py[1]))
    probs = softmax_manual(dot_manual(h2, W_py[2], b_py[2]))
    idx = probs.index(max(probs))
    signal_class = CLASSES_PY[idx]
    stress_level = stress_level_map.get(signal_class, "unknown")
```

```
    return signal_class, stress_level, probs[idx], probs

X_test_raw = scaler.inverse_transform(Xte)
mismatches = 0

hdr = f"{'Win':>3}  {'Actual':15}  {'sklearn':15}  {'Python':15}  {'St
print(hdr)
print("—" * len(hdr))

for i in range(len(X_test_raw)):
    raw = X_test_raw[i].tolist()
    sk_lbl = clf.predict([Xte[i]])[0]
    py_lbl, py_level, py_conf, py_prob = predict_manual(raw)
    ok = "✓" if sk_lbl == py_lbl else "✗"
    if sk_lbl != py_lbl:
        mismatches += 1
    print(f"{i:>3}  {y_test[i]:15}  {str(sk_lbl):15}  {py_lbl:15}  {py

print()
if mismatches == 0:
    print("✓ All predictions match — exported weights reproduce sklear
else:
    print(f"✗ {mismatches} mismatch(es) — check scaler parameters or w

print()
raw0 = X_test_raw[0].tolist()
py0_lbl, py0_level, py0_conf, _ = predict_manual(raw0)
print("=== Core2 spot-check ===")
print(f"raw_features = {[round(v, 4) for v in raw0]}")
print(f"Expected signal class: {py0_lbl}")
print(f"Expected stress level: {py0_level}")
print(f"Expected confidence  : {py0_conf:.4f}")
```

```
Win  Actual           sklearn          Python           Stress Level
✓?
──────────────────────────────────────────────────────────────────────
──────────────────
  0  tapping          tapping          tapping          level_2_motion_
stress     ✓
  1  sucking_teeth    sucking_teeth    sucking_teeth    level_1_sound_s
tress      ✓
  2  sucking_teeth    humming          humming          level_1_sound_s
tress      ✓
  3  vocal_distress   vocal_distress   vocal_distress   level_1_sound_s
tress      ✓
  4  shaking          shaking          shaking          level_2_motion_
stress     ✓
  5  humming          sucking_teeth    sucking_teeth    level_1_sound_s
tress      ✓
  6  fidgeting        tapping          tapping          level_2_motion_
stress     ✓
  7  fidgeting        fidgeting        fidgeting        level_2_motion_
stress     ✓
  8  humming          humming          humming          level_1_sound_s
tress      ✓
  9  vocal_distress   vocal_distress   vocal_distress   level_1_sound_s
tress      ✓
 10  tapping          tapping          tapping          level_2_motion_
stress     ✓
 11  vocal_distress   vocal_distress   vocal_distress   level_1_sound_s
tress      ✓
 12  baseline         baseline         baseline         level_0_no_stre
ss         ✓
 13  baseline         baseline         baseline         level_0_no_stre
ss         ✓
 14  sucking_teeth    humming          humming          level_1_sound_s
tress      ✓
 15  shaking          shaking          shaking          level_2_motion_
stress     ✓
 16  shaking          shaking          shaking          level_2_motion_
stress     ✓
 17  humming          humming          humming          level_1_sound_s
tress      ✓

✓ All predictions match — exported weights reproduce sklearn prediction
s.

=== Core2 spot-check ===
raw_features = [0.0, 0.0, 0.0, 0.0, 0.0, 1.0643, 0.0914, 1.2788, 0.332
6, 1.141, 75.8167, 31.6645, 156.694, 131.2632, 6750.8067]
Expected signal class: tapping
Expected stress level: level_2_motion_stress
Expected confidence  : 0.9808
```